Three organizations of Mapping techniques

Direct mapping

Fully Associative Mapping

Set-Associative Mapping

Direct Mapping

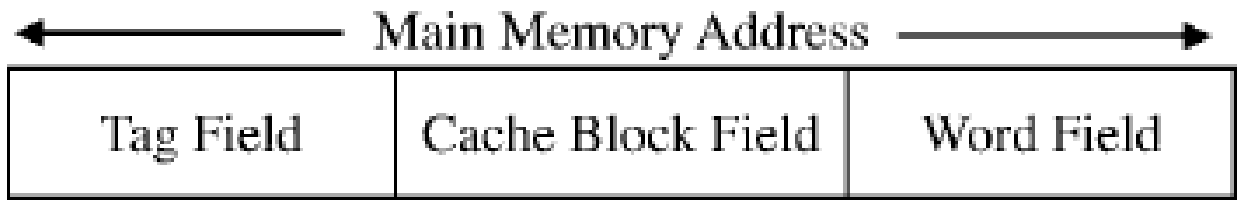
This is the simplest among the three techniques. Its simplicity stems from the fact that it places an incoming main memory block into a specific fixed cache block location.

The placement is done based on a fixed relation between the incoming block number, i,
the cache block number, j,
and the number of cache blocks, N:

$$j = i \bmod N \quad ??$$

Example 1: Consider, the case of a main memory consisting of 4K blocks, a cache memory consisting of 128 blocks, and a block size of 16 words.

According to the direct-mapping technique the MMU interprets the address issued by the processor by dividing the address into three fields



1- Word field = $\log_2 B$, B is the size of the block in words

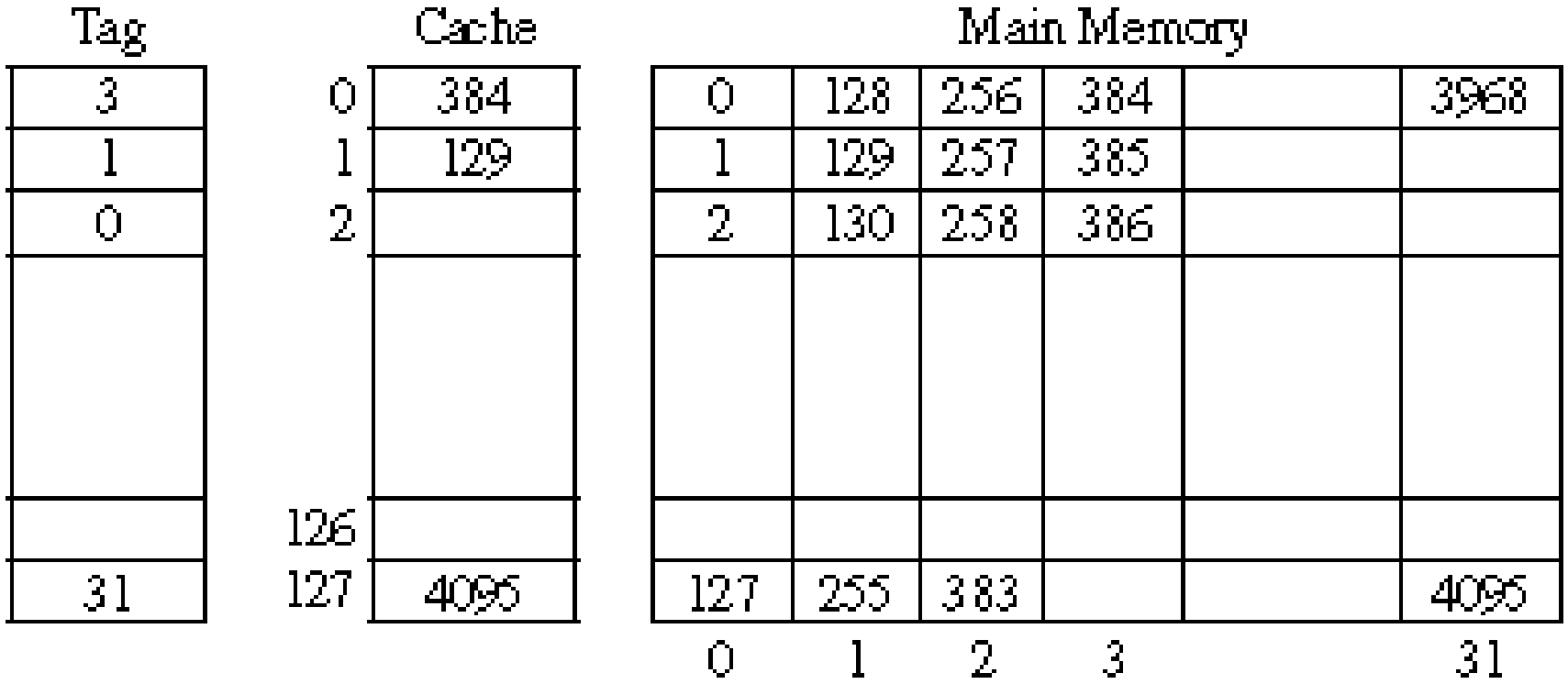
2- Block field = $\log_2 N$, N is the size in the cache in blocks (lines)

3- Tag field = $\log_2 (M/N)$, M is the size of the main memory in blocks

4- the number of bits in the main memory address = $\log_2 (B \times M)$

As the figure shows, there are a total of 32 main memory blocks that map to a given cache block. For example, main memory blocks 0, 128, 256, 384, . . . , 3968 map to cache block 0. We therefore call the direct-mapping technique a many-to-one mapping technique.

4k=4096
4096*16=65536=16 bit address of main memory



The MMU protocol steps are:

1. Use the Block field to determine the cache block that should contain the element requested by the processor. The Block field is used directly to determine the cache block sought, hence the name of the technique: direct-mapping.

2. Check the corresponding Tag memory to see whether there is a match between its content and that of the Tag field. A match between the two indicates that the targeted cache block determined in step 1 is currently holding the requested element by the processor, that is, a cache hit.

3. Among the elements contained in the cache block, the targeted element can be selected using the Word field.

4. If in step 2, no match is found, then this indicates a cache miss. Therefore, the required block has to be brought from the main memory, deposited (يوضع)in the cache, and the targeted element is made available to the processor. The cache Tag memory and the cache block memory have to be updated accordingly.

Compute the above four parameters for our Example

1- Word field = $\log_2 B = \log_2(16) = \log_2(2^4) = 4$ bits

2- Block field =Line fields = $\log_2 N = \log_2(128) = \log_2(2^7) = 7$ bits

3- Tag field = $\log_2(M/N) = \log_2(2^2 \times 2^{10} / 2^7) = \log_2(2^5) = 5$ bits

4- the number of bits in the main memory address = $\log_2(B \times M) = = \log_2(2^4 \times 2^{12}) = \log_2(2^{16}) =$ 16 bits the address

Here, the "Word" field selects one from among the 16 addressable words in a line.
The block "Line" field defines the cache line where this memory line should reside.
The "Tag" field of the address is then compared with that cache line's 5-bit tag to determine whether there is a <u>hit or a miss.</u>

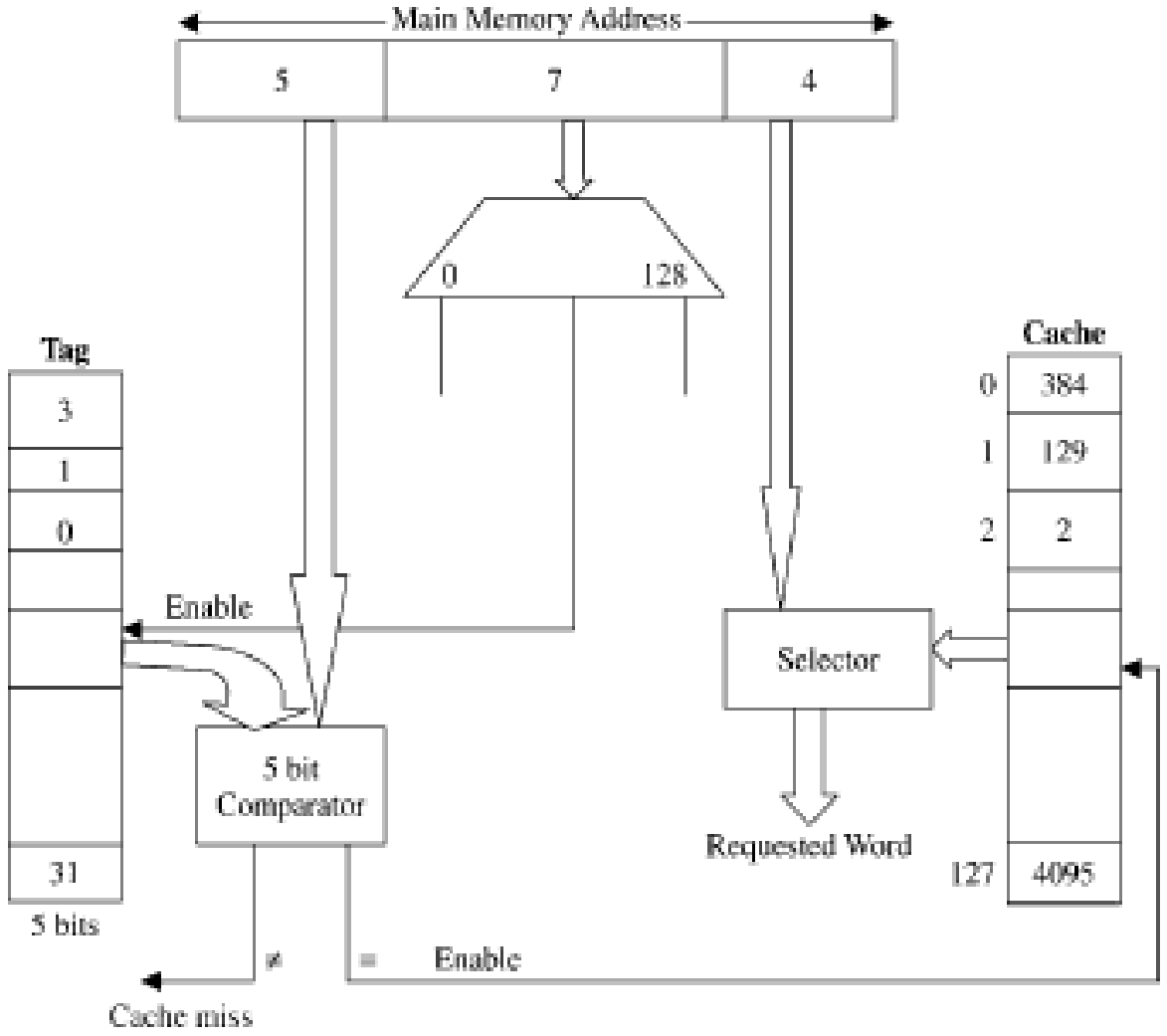
If there's a miss, we need to swap out the memory line that occupies that position in the cache and replace it with the desired memory line.

E.g., Suppose we want to read or write a word at the address 357A, whose 16 bits are 00110 1010111 1010.
This translates to Tag = 6, line = 87, and Word = 10 (all in decimal).

If line 87 in the cache has the same tag (6), then memory address 357A is in the cache.

Otherwise, a miss has occurred and the contents of cache line 87 must be replaced by the memory line 001101010111 = 855 before the read or write is executed.

Direct mapping is the most efficient cache mapping scheme, but it is also the least effective in its utilization of the cache - that is, it may leave some cache lines unused.



Summery of direct: Use tag to see if a desired word is in cache

– It there is no match, the block containing the required word must first be read from the memory

Ex: address is A815

10101	0000001	0101
Tag	Block #	Word

a. Check if cache has tag 10101 for block 1
match -> hit; different -> miss, load the corresponding block

b. Access word 5 of the block

Fully Associative Mapping

The MMU interprets the address issued by the processor by dividing it into two fields as

1- Word field = $\log_2 B$, B is the size of the block in words

2- Tag field = $\log_2 (M)$, M is the size of the main memory in blocks

3- the number of bits in the main memory address = $\log_2 (B \times M)$

we can now proceed to explain the protocol used by the MMU to satisfy a request, the protocol steps are:

1. Use the Tag field to search in the Tag memory for a match with any of the tags stored.

2. A match in the tag memory indicates that the corresponding targeted cache block determined in step 1 is currently holding the main memory element requested by the processor, that is, a cache hit.

3. Among the elements contained in the cache block, the targeted element can be selected using the Word field.

4. If in step 2, no match is found, then this indicates a cache miss. Therefore, the required block has to be brought from the main memory, deposited in the first available cache block, and the targeted element (word) is made available to the processor. The cache Tag memory and the cache block memory have to be updated accordingly.

Return to our example:

1- Word field = $\log_2 B = \log_2(16) = \log_2(2^4) = 4$ bits

2- Tag field = $\log_2(M) = \log_2(2^2 \times 2^{10}) = \log_2(2^{12}) = 12$ bits

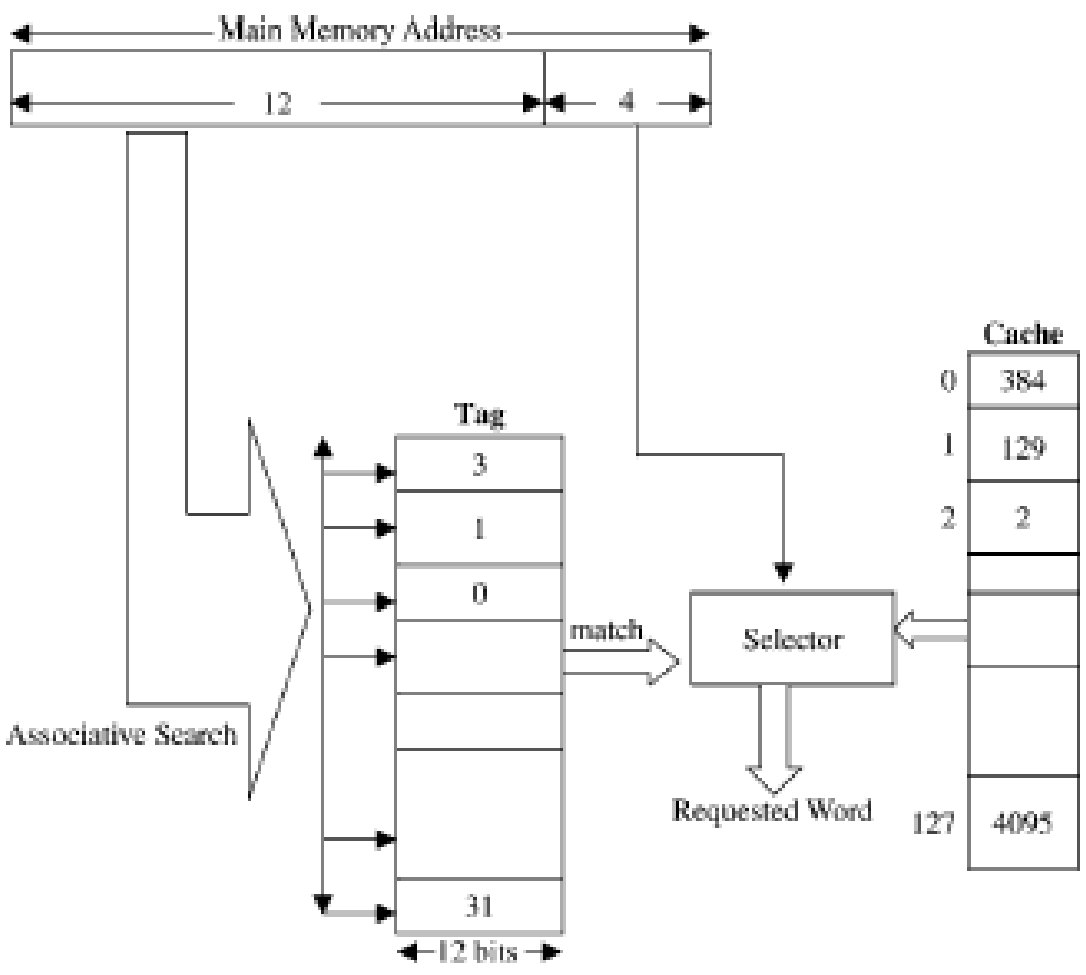
3- the number of bits in the main memory address = $\log_2(B \times M) = = \log_2(2^4 \times 2^{12}) = \log_2(2^{16}) = 16$ bits

Here, the "Tag" field identifies one of the $2^{12} = 4096$ memory lines; all the cache tags are searched to find out whether or not the Tag field matches one of the cache tags. If so, we have a hit, and if not there's a miss and we need to replace one of the cache lines by this line before reading or writing into the cache. (The "Word" field again selects one from among 16 addressable words (bytes) within the line.)

For example, suppose again that we want to read or write a word at the address 357A, whose 16 bits are 0011010101111010. Under associative mapping, this translates to Tag = 855 and Word = 10 (in decimal).

So we search all of the 128 cache tags to see if any one of them will match with 855. If not, there's a miss and we need to replace one of the cache lines with line 855 from memory before completing the read or write.

The search of all 128 tags in the cache is time-consuming. However, the cache is fully utilized since none of its lines will be unused prior to a miss (recall that direct mapping may detect a miss even though the cache is not completely full of active lines).



Set-Associative Mapping

In the set-associative mapping technique, the cache is divided into a number of sets. Each set consists of a number of blocks. A given main memory block maps to a specific cache set based on the following:

1- Word field = $\log_2 B$, B is the size of the block in words

2- Set field = $\log_2 S$, S is the number of sets in the cache,

S= N/Bs, where N is the number of cache blocks and Bs is the number of blocks per set.

3- Tag field = $\log_2 (M/S)$, M is the size of the main memory in blocks

4- the number of bits in the main memory address = $\log_2 (B \times M)$

Return to our example:

Assume that the system uses set-associative mapping with four blocks per set.

First: S= N/Bs = 128/4 = 32 sets

1- Word field = $\log_2 B = \log_2(16) = \log_2(2^4) = 4$ bits

2- set field= $\log_2(32) = \log_2(2^5) = 5$ bits

3- Tag field= $\log_2(2^2 \times 2^{10}/32) = 7$ bits

3- the number of bits in the main memory address = $\log_2(B \times M) = = \log_2(2^4 \times 2^{12}) = \log_2(2^{16}) = 16$ bits

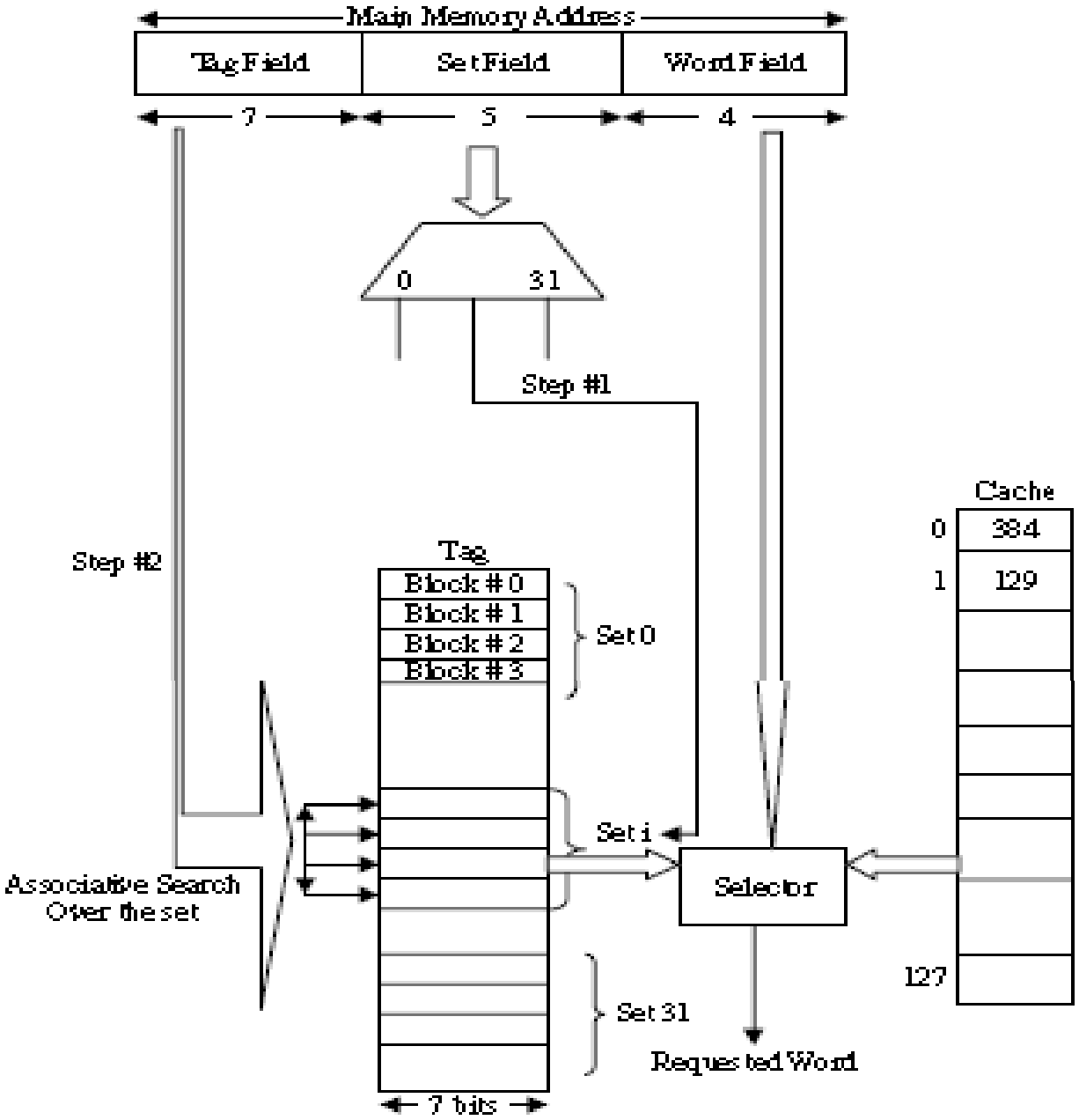
the protocol used by the MMU to satisfy a request made by the processor for accessing a given element. the steps of this protocol is:

1. Use the Set field (5 bits) to determine (directly) the specified set (1 of the 32 sets).

2. Use the Tag field to find a match with any of the (four) blocks in the determined set. A match in the tag memory indicates that the specified set determined in step 1 is currently holding the targeted block, that is, a cache hit.

3. Among the 16 words (elements) contained in hit cache block, the requested word is selected using a selector with the help of the Word field.

4. If in step 2, no match is found, then this indicates a cache miss. Therefore, the required block has to be brought from the main memory, deposited in the specified set first, and the targeted element (word) is made available to the processor. The cache Tag memory and the cache block memory have to be updated accordingly.

E.g., Again suppose we want to read or write a word at the memory address 357A, whose 16 bits are 0011010101111010.

Under set-associative mapping, this translates to Tag = 26, Set = 23, and Word = 10 (all in decimal).
So we search tags in cache set 23 to see if either one matches tag 26. If so, we have a hit. Otherwise, one of these blocks must be replaced by the memory line being addressed (good old line 855) before the read or write can be executed.